

Complete Database Schema

Sports Match-Prediction Application

Sprint 1 - Backend Planning & Database Design

Proposed stack: FastAPI + PostgreSQL + SQLAlchemy + Alembic

Status: Team-review schema draft

1. Purpose and Scope

This schema defines how the backend can store sports, leagues, seasons, teams, scheduled matches, final results, machine-generated match predictions, optional user picks, and external sports-data identifiers. It is designed to work cleanly with FastAPI, PostgreSQL, SQLAlchemy, and Alembic while remaining small enough for a capstone implementation.

1.1 Core Design Principles

Principle	How it applies
Normalized core data	Sports, leagues, seasons, teams, and matches are stored once and referenced by foreign keys.
Separate model output from user picks	AI predictions and user-submitted picks have different tables so they can evolve independently.
External API independence	Third-party IDs are stored in a mapping table instead of becoming the application's primary keys.
Traceable model versions	Every generated prediction points to the exact model version that created it.
Future-ready, not overbuilt	Supporting tables are included where they solve a real integration or traceability problem, but payment/betting tables are intentionally excluded.

2. Entity-Relationship Diagram

The diagram below shows the complete proposed relational structure. Solid relationships represent foreign-key dependencies; season membership and external mappings provide flexibility without duplicating core entities.

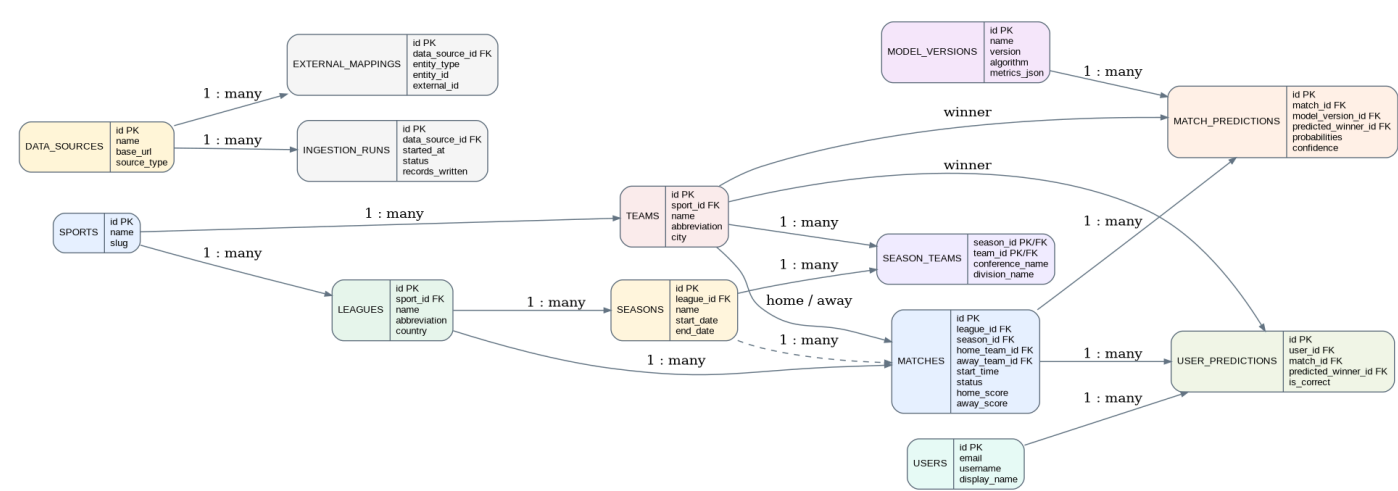


Figure 1. Complete database ERD for the proposed backend schema.

3. Table Summary

Table	Purpose	Layer
sports	Top-level sport categories such as Basketball, Football, Baseball, Hockey, Soccer.	Core
leagues	Competitions/leagues within a sport such as NBA, NFL, MLB, NHL, EPL.	Core
seasons	League seasons used to separate schedules and historical data.	Core
teams	Canonical team records shared across seasons.	Core
season_teams	Many-to-many season membership for teams.	Core
matches	Scheduled, in-progress, final, postponed, or cancelled matches.	Core
model_versions	Metadata for each prediction model/version.	Prediction
match_predictions	AI/model output for a match.	Prediction
users	Optional application account profiles.	Optional MVP
user_predictions	Optional user picks for individual matches.	Optional MVP
data_sources	External sports-data providers or manual/file sources.	Integration
external_mappings	Maps internal entities to provider-specific IDs.	Integration
ingestion_runs	Tracks each external data import/sync attempt.	Integration

4. Detailed Schema

4.1 sports

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Internal sport identifier.
name	VARCHAR(80)	NOT NULL, UNIQUE	Display name.
slug	VARCHAR(80)	NOT NULL, UNIQUE	Stable URL/API key such as basketball.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update timestamp.

4.2 leagues

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Internal league identifier.

Column	Type	Constraint	Purpose
sport_id	BIGINT	FK -> sports.id	Parent sport.
name	VARCHAR(120)	NOT NULL	League/competition name.
abbreviation	VARCHAR(30)	NULL	Short code such as NBA or EPL.
country	VARCHAR(80)	NULL	Country/region if applicable.
active	BOOLEAN	DEFAULT TRUE	Allows leagues to be hidden without deleting data.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update timestamp.

4.3 seasons

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Internal season identifier.
league_id	BIGINT	FK -> leagues.id	Owning league.
name	VARCHAR(60)	NOT NULL	Season label, e.g. 2026-27.
start_date	DATE	NULL	Season start.
end_date	DATE	NULL	Season end.
is_current	BOOLEAN	DEFAULT FALSE	Current season flag.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update timestamp.

4.4 teams

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Internal team identifier.
sport_id	BIGINT	FK -> sports.id	Sport for the team.
name	VARCHAR(140)	NOT NULL	Canonical team name.
abbreviation	VARCHAR(20)	NULL	Short code.
city	VARCHAR(100)	NULL	Home city/region.
logo_url	TEXT	NULL	Optional team logo URL.
active	BOOLEAN	DEFAULT TRUE	Active/inactive flag.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update timestamp.

4.5 season_teams

Column	Type	Constraint	Purpose
season_id	BIGINT	PK, FK -> seasons.id	Season membership.
team_id	BIGINT	PK, FK -> teams.id	Team membership.

Column	Type	Constraint	Purpose
conference_name	VARCHAR(80)	NULL	Optional conference grouping.
division_name	VARCHAR(80)	NULL	Optional division grouping.

4.6 matches

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Internal match identifier.
league_id	BIGINT	FK -> leagues.id	League for the match.
season_id	BIGINT	FK -> seasons.id, NULL	Season for the match.
home_team_id	BIGINT	FK -> teams.id	Home team.
away_team_id	BIGINT	FK -> teams.id	Away team.
start_time	TIMESTAMPTZ	NOT NULL	Scheduled match time.
status	VARCHAR(30)	CHECK	scheduled/in_progress/final/postponed/cancelled.
home_score	INTEGER	NULL	Final/current home score.
away_score	INTEGER	NULL	Final/current away score.
neutral_site	BOOLEAN	DEFAULT FALSE	Neutral venue indicator.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update timestamp.

4.7 model_versions

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Model-version identifier.
name	VARCHAR(120)	NOT NULL	Model family/name.
version	VARCHAR(50)	NOT NULL	Version label.
algorithm	VARCHAR(120)	NULL	Algorithm/model type.
training_cutoff	TIMESTAMPTZ	NULL	Latest data used for training.
metrics_json	JSONB	NULL	Evaluation metrics such as accuracy, precision, etc.
active	BOOLEAN	DEFAULT TRUE	Whether this model is currently available.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.

4.8 match_predictions

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Generated prediction identifier.
match_id	BIGINT	FK -> matches.id	Predicted match.
model_version_id	BIGINT	FK -> model_versions.id	Model that produced the result.

Column	Type	Constraint	Purpose
predicted_winner_id	BIGINT	FK -> teams.id	Predicted winning team.
home_win_probability	NUMERIC(5,4)	0..1	Home win probability.
away_win_probability	NUMERIC(5,4)	0..1	Away win probability.
confidence	NUMERIC(5,4)	0..1	Optional model confidence value.
generated_at	TIMESTAMPTZ	DEFAULT NOW()	Prediction creation time.
explanation_json	JSONB	NULL	Optional factors/explanation payload.

4.9 users

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	User identifier.
email	VARCHAR(254)	NOT NULL, UNIQUE	Account email.
username	VARCHAR(80)	UNIQUE, NULL	Optional username.
display_name	VARCHAR(120)	NULL	Display name.
is_active	BOOLEAN	DEFAULT TRUE	Account state.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update timestamp.

4.10 user_predictions

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	User-pick identifier.
user_id	BIGINT	FK -> users.id	User making the pick.
match_id	BIGINT	FK -> matches.id	Selected match.
predicted_winner_id	BIGINT	FK -> teams.id	User-selected winner.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Pick time.
is_correct	BOOLEAN	NULL until final	Resolved after match is final.

4.11 data_sources

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Provider identifier.
name	VARCHAR(100)	NOT NULL, UNIQUE	Provider/source name.
base_url	TEXT	NULL	Provider base URL.
source_type	VARCHAR(30)	CHECK	api/manual/file.
active	BOOLEAN	DEFAULT TRUE	Whether imports are enabled.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.

4.12 external_mappings

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Mapping identifier.
data_source_id	BIGINT	FK -> data_sources.id	Source that owns the external ID.
entity_type	VARCHAR(30)	CHECK	league/season/team/match.
entity_id	BIGINT	NOT NULL	Internal entity ID; validated by service logic.
external_id	VARCHAR(160)	NOT NULL	Provider-specific identifier.
created_at	TIMESTAMPTZ	DEFAULT NOW()	Creation timestamp.

4.13 ingestion_runs

Column	Type	Constraint	Purpose
id	BIGSERIAL	PK	Import-run identifier.
data_source_id	BIGINT	FK -> data_sources.id	Source being synchronized.
started_at	TIMESTAMPTZ	DEFAULT NOW()	Run start.
completed_at	TIMESTAMPTZ	NULL	Run completion.
status	VARCHAR(20)	CHECK	running/success/failed.
records_read	INTEGER	DEFAULT 0	Records returned by source.
records_written	INTEGER	DEFAULT 0	Rows inserted/updated.
error_message	TEXT	NULL	Failure details for debugging.

5. Relationship Rules

Relationship	Rule
Sport -> League	One sport can contain many leagues; every league belongs to one sport.
League -> Season	A league can have many seasons; each season belongs to one league.
Season <-> Team	A team can appear in many seasons and a season contains many teams through season_teams.
League/Season -> Match	Every match belongs to a league and can optionally point to a season.
Team -> Match	Every match references two different teams: home_team_id and away_team_id.
Match -> Model Prediction	A match can have predictions from multiple model versions; one result per model version.
User -> User Prediction	A user can make many picks, but only one pick per match.

Relationship	Rule
Data Source -> Mapping	External IDs are mapped to internal IDs so providers can be swapped without redesigning core tables.
Data Source -> Ingestion Run	Each import attempt is logged for traceability and debugging.

6. Important Constraints and Validation

Constraint	Reason
Different teams per match	matches.home_team_id must not equal matches.away_team_id.
Known match status	Only scheduled, in_progress, final, postponed, or cancelled are accepted.
Non-negative scores	Scores must be NULL or ≥ 0 .
Probabilities	Prediction probability/confidence values must be between 0 and 1.
One model result per match/version	UNIQUE(match_id, model_version_id) prevents duplicate model outputs.
One user pick per match	UNIQUE(user_id, match_id) prevents duplicate user picks.
Provider ID uniqueness	External IDs are unique within a provider and entity type.
Season date validity	end_date cannot be earlier than start_date.

7. Recommended Indexes

Index	Primary use
matches(start_time)	Upcoming-match queries and date filters.
matches(league_id, start_time)	League schedule pages.
matches(home_team_id), matches(away_team_id)	Team schedule/history queries.
match_predictions(match_id)	Retrieve model output for one match.
user_predictions(user_id)	User history/dashboard.
user_predictions(match_id)	Resolve picks after final score.
ingestion_runs(data_source_id, started_at DESC)	Provider sync history and debugging.

8. How Data Moves Through the Schema

Step	What happens
1. Sync sports data	A scheduled backend job or manual command reads an external sports API. A row in ingestion_runs records the attempt.
2. Resolve provider IDs	external_mappings translates provider league/team/match IDs into the application's internal PostgreSQL IDs.

Step	What happens
3. Upsert core records	sports, leagues, seasons, teams, season_teams, and matches are inserted or updated.
4. Generate prediction	The prediction service loads the required match/team data and a model_versions record identifies the model being used.
5. Store model output	match_predictions stores winner probability, confidence, timestamp, and optional explanation data.
6. Serve through FastAPI	Frontend endpoints query PostgreSQL through SQLAlchemy and return normalized JSON.
7. Resolve results	When a match becomes final, scores are updated. Optional user_predictions.is_correct can then be calculated.

9. Proposed API-to-Table Mapping

Endpoint	Primary tables	Purpose
GET /sports	sports	List supported sports.
GET /leagues	leagues, sports	List/filter leagues.
GET /leagues/{id}/teams	season_teams, teams, seasons	Current season roster of teams.
GET /matches	matches, teams, leagues	Upcoming/recent matches.
GET /matches/{id}	matches, teams, leagues	Detailed match record.
GET /matches/{id}/prediction	match_predictions, model_versions	Latest/current model prediction.
POST /users/{id}/predictions	user_predictions	Optional user pick submission.
GET /users/{id}/predictions	user_predictions, matches, teams	Optional user pick history.

10. Sprint 1 Implementation Recommendation

For Sprint 1, the entire schema can be documented now, but implementation should begin with the core path needed to prove the backend works end-to-end. This reduces risk while preserving the complete design for later sprints.

Phase	Tables
Implement now	sports, leagues, seasons, teams, season_teams, matches, data_sources, external_mappings
Add when prediction model is ready	model_versions, match_predictions
Add if accounts/user picks are confirmed	users, user_predictions
Add when automated data syncing begins	ingestion_runs

11. Team Review Questions

- Will the MVP require user accounts, or should users only view model-generated predictions?
- Will the first model predict only a winner, or also expose probability/confidence?
- Which leagues are required for the first implemented vertical slice?

- Which external sports-data provider will be used, and what IDs/data does it expose?
- Do we want to keep season membership flexible through season_teams, or simplify teams to one league for the first prototype?
- How often should match schedules and final scores be synchronized?

12. Deliverables Included

This package includes the Word schema document and a PostgreSQL DDL file that can be adapted into SQLAlchemy models and Alembic migrations after team approval.

File	Use
Complete_Database_Schema_Sprint1.docx	Team presentation, design review, and capstone documentation.
sports_prediction_schema.sql	Concrete PostgreSQL reference for backend implementation.